In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv( r"C:\Users\ASUS\OneDrive\Desktop\Cafe_Sales_Raw-1.csv")

print(df)
```

```
     Transaction ID      Item Quantity Price Per Unit Total Spent  \
0       TXN_1961373    Coffee        2              2           4
1       TXN_4977031      Cake        4              3          12
2       TXN_4271903    Cookie        4              1       ERROR
3       TXN_7034554     Salad        2              5          10
4       TXN_3160411    Coffee        2              2           4
...             ...       ...      ...            ...         ...
9995    TXN_7672686    Coffee        2              2           4
9996    TXN_9659401       NaN        3            NaN           3
9997    TXN_5255387    Coffee        4              2           8
9998    TXN_7695629    Cookie        3            NaN           3
9999    TXN_6170729  Sandwich        3              4          12

      Payment Method  Location Transaction Date
0        Credit Card  Takeaway       08-09-2023
1               Cash  In-store       16-05-2023
2        Credit Card  In-store       19-07-2023
3            UNKNOWN   UNKNOWN       27-04-2023
4     Digital Wallet  In-store       11-06-2023
...              ...       ...              ...
9995             NaN   UNKNOWN       30-08-2023
9996  Digital Wallet       NaN       02-06-2023
9997  Digital Wallet       NaN       02-03-2023
9998  Digital Wallet       NaN       02-12-2023
9999            Cash  In-store       07-11-2023

[10000 rows x 8 columns]
```

In [2]:
```
df
```

Out[2]:

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transac
0	TXN_1961373	Coffee	2	2	4	Credit Card	Takeaway	08-09-2
1	TXN_4977031	Cake	4	3	12	Cash	In-store	16-05-2
2	TXN_4271903	Cookie	4	1	ERROR	Credit Card	In-store	19-07-2
3	TXN_7034554	Salad	2	5	10	UNKNOWN	UNKNOWN	27-04-2
4	TXN_3160411	Coffee	2	2	4	Digital Wallet	In-store	11-06-2
...	...	...	...	...	...	...	...	
9995	TXN_7672686	Coffee	2	2	4	NaN	UNKNOWN	30-08-2
9996	TXN_9659401	NaN	3	NaN	3	Digital Wallet	NaN	02-06-2
9997	TXN_5255387	Coffee	4	2	8	Digital Wallet	NaN	02-03-2
9998	TXN_7695629	Cookie	3	NaN	3	Digital Wallet	NaN	02-12-2
9999	TXN_6170729	Sandwich	3	4	12	Cash	In-store	07-11-2

10000 rows × 8 columns

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   Transaction ID    10000 non-null  object
 1   Item              9667 non-null   object
 2   Quantity          9862 non-null   object
 3   Price Per Unit    9821 non-null   object
 4   Total Spent       9827 non-null   object
 5   Payment Method    7421 non-null   object
 6   Location          6735 non-null   object
 7   Transaction Date  9841 non-null   object
dtypes: object(8)
memory usage: 625.1+ KB
```

In [4]:
```
# Numeric conversion
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")

df["Price Per Unit"] = pd.to_numeric(
    df["Price Per Unit"], errors="coerce"
)

df["Total Spent"] = pd.to_numeric(
    df["Total Spent"], errors="coerce"
)
```

```
# Date conversion
df["Transaction Date"] = pd.to_datetime(
    df["Transaction Date"], errors="coerce"
)

# Category conversion
df["Payment Method"] = df["Payment Method"].astype("category")
df["Location"] = df["Location"].astype("category")
```

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype         
---  ------            --------------  -----         
 0   Transaction ID    10000 non-null  object        
 1   Item              9667 non-null   object        
 2   Quantity          9521 non-null   float64       
 3   Price Per Unit    9467 non-null   float64       
 4   Total Spent       9498 non-null   float64       
 5   Payment Method    7421 non-null   category      
 6   Location          6735 non-null   category      
 7   Transaction Date  3742 non-null   datetime64[ns]
dtypes: category(2), datetime64[ns](1), float64(3), object(2)
memory usage: 488.8+ KB
```

In [6]:
```
df.describe()
```

Out[6]:

	Quantity	Price Per Unit	Total Spent	Transaction Date
count	9521.000000	9467.000000	9498.000000	3742
mean	3.028463	2.949984	8.924352	2023-06-22 03:36:16.162479872
min	1.000000	1.000000	1.000000	2023-01-01 00:00:00
25%	2.000000	2.000000	4.000000	2023-04-02 00:00:00
50%	3.000000	3.000000	8.000000	2023-06-12 00:00:00
75%	4.000000	4.000000	12.000000	2023-09-11 00:00:00
max	5.000000	5.000000	25.000000	2023-12-12 00:00:00
std	1.419007	1.278450	6.009919	NaN

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
Transaction ID       0
Item               333
Quantity           479
Price Per Unit     533
Total Spent        502
Payment Method     2579
Location           3265
Transaction Date   6258
dtype: int64
```

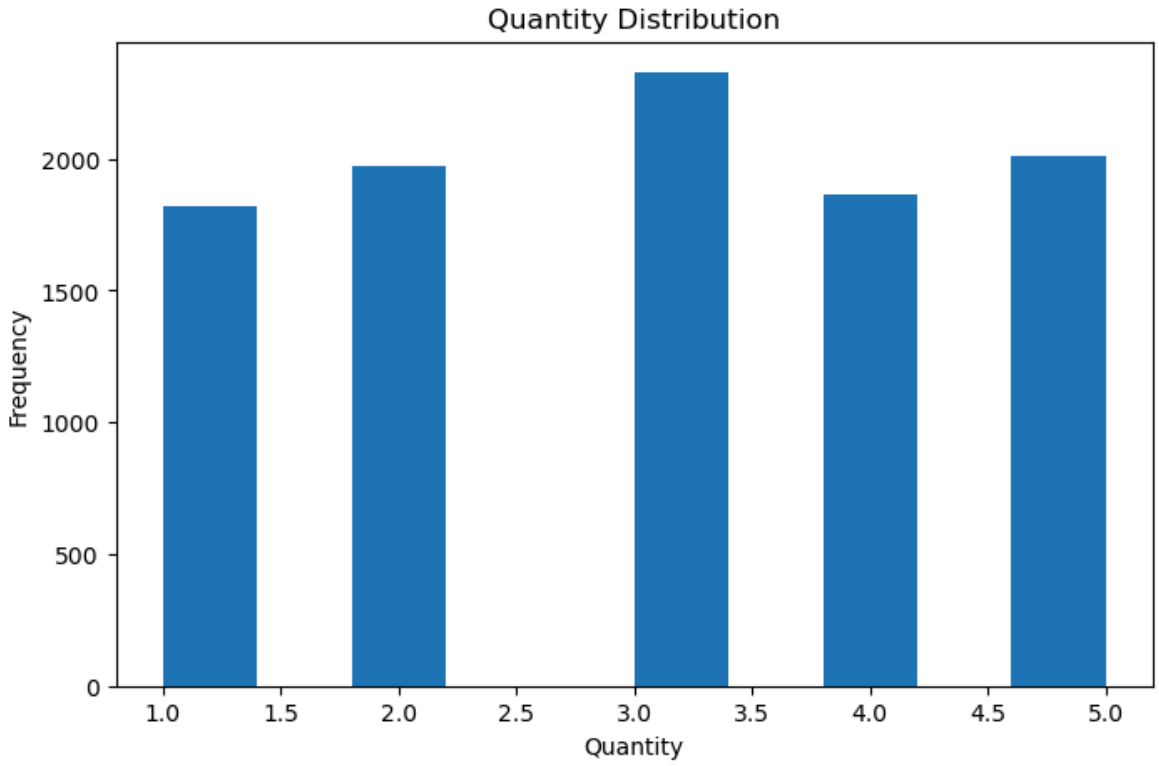
In [10]:
```
print(df.duplicated().sum())
df.drop_duplicates(inplace=True)
```

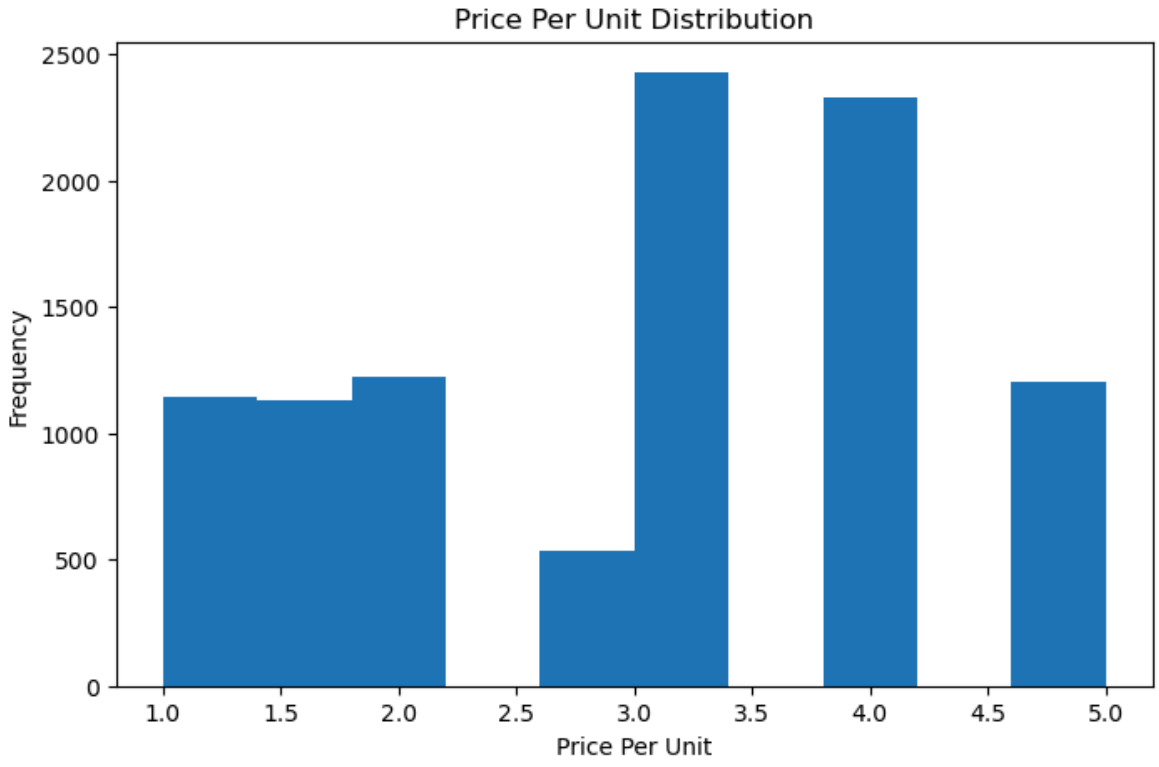
```
0
```

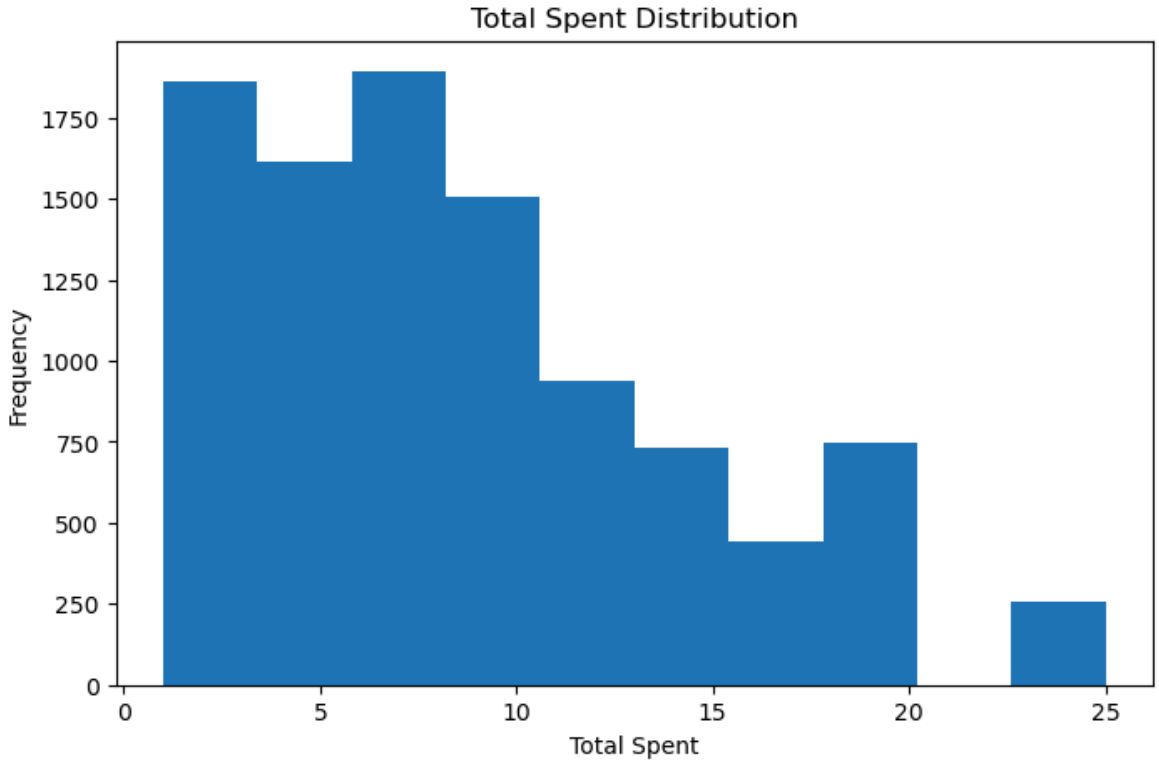
In [23]:
```
# 3. Handle UNKNOWN and ERROR values

# Mode values
mod_item = df["Item"].mode()[0]
mod_payment = df["Payment Method"].mode()[0]
mod_location = df["Location"].mode()[0]

# Mean values
mean_qty = df["Quantity"].mean()
mean_price = df["Price Per Unit"].mean()
mean_total = df["Total Spent"].mean()
mean_date = df["Transaction Date"].mean()

# Median values
med_qty = df["Quantity"].median()
med_price = df["Price Per Unit"].median()
med_total = df["Total Spent"].median()
med_date = df["Transaction Date"].median()

# Replace UNKNOWN

df["Item"] = df["Item"].replace("UNKNOWN", mod_item)

df["Quantity"] = df["Quantity"].replace("UNKNOWN", mean_qty)

df["Price Per Unit"] = df["Price Per Unit"].replace("UNKNOWN", mean_price)

df["Total Spent"] = df["Total Spent"].replace("UNKNOWN", mean_total)

df["Payment Method"] = df["Payment Method"].replace("UNKNOWN", mod_payment)

df["Location"] = df["Location"].replace("UNKNOWN", mod_location)

# Replace ERROR

df["Item"] = df["Item"].replace("ERROR", mod_item)

df["Quantity"] = df["Quantity"].replace("ERROR", med_qty)

df["Price Per Unit"] = df["Price Per Unit"].replace("ERROR", med_price)

df["Total Spent"] = df["Total Spent"].replace("ERROR", med_total)

df["Payment Method"] = df["Payment Method"].replace("ERROR", mod_payment)

df["Location"] = df["Location"].replace("ERROR", mod_location)
# 4. Fill Remaining Missing Values

df["Item"] = df["Item"].fillna(mod_item)

df["Quantity"] = df["Quantity"].fillna(mean_qty)

df["Price Per Unit"] = df["Price Per Unit"].fillna(mean_price)

df["Total Spent"] = df["Total Spent"].fillna(mean_total)
```

```
df["Payment Method"] = df["Payment Method"].fillna(mod_payment)

df["Location"] = df["Location"].fillna(mod_location)

df["Transaction Date"] = df["Transaction Date"].fillna(mean_date)
# 5. Remove Duplicate Records

print("\nDuplicate Rows Before:")
print(df.duplicated().sum())

df.drop_duplicates(inplace=True)

print("\nDuplicate Rows After:")
print(df.duplicated().sum())

# 6. Correlation Analysis

print("\nCorrelation Matrix:\n")
print(df.corr(numeric_only=True))

# 7. Visualizations

# Quantity Distribution

plt.figure(figsize=(8,5))
plt.hist(df["Quantity"])
plt.title("Quantity Distribution")
plt.xlabel("Quantity")
plt.ylabel("Frequency")
plt.show()

# Price Per Unit Distribution

plt.figure(figsize=(8,5))
plt.hist(df["Price Per Unit"])
plt.title("Price Per Unit Distribution")
plt.xlabel("Price Per Unit")
plt.ylabel("Frequency")
plt.show()

# Total Spent Distribution

plt.figure(figsize=(8,5))
plt.hist(df["Total Spent"])
plt.title("Total Spent Distribution")
plt.xlabel("Total Spent")
plt.ylabel("Frequency")
plt.show()
```

```
Duplicate Rows Before:
0

Duplicate Rows After:
0

Correlation Matrix:

                Quantity  Price Per Unit  Total Spent
Quantity        1.000000        0.005756     0.669235
Price Per Unit  0.005756        1.000000     0.612996
Total Spent     0.669235        0.612996     1.000000
```







In [9]:
```
df.isnull().sum()
```

Out[9]:
```
Transaction ID      0
Item                0
Quantity            0
Price Per Unit      0
Total Spent         0
Payment Method      0
Location            0
Transaction Date    0
dtype: int64
```

In [11]:
```
df
```

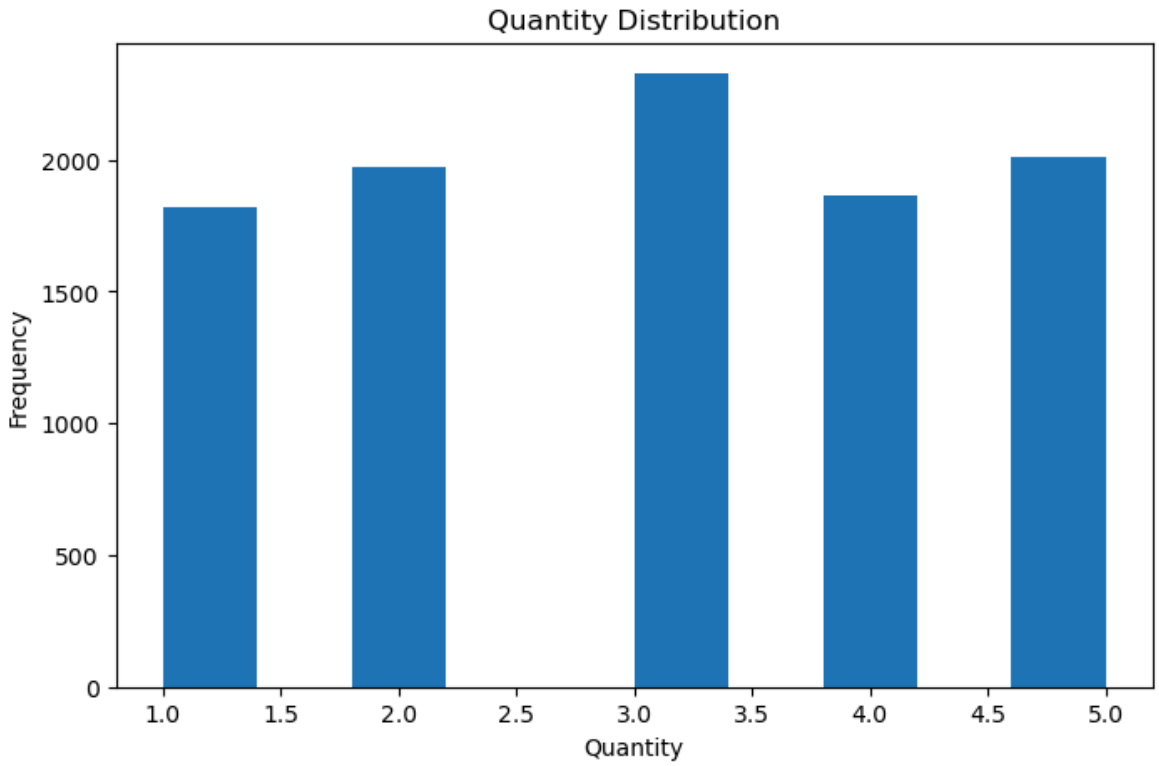
Out[11]:

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Tra
0	TXN_1961373	Coffee	2.0	2.000000	4.000000	Credit Card	Takeaway	00:00
1	TXN_4977031	Cake	4.0	3.000000	12.000000	Cash	In-store	03:36
2	TXN_4271903	Cookie	4.0	1.000000	8.924352	Credit Card	In-store	03:36
3	TXN_7034554	Salad	2.0	5.000000	10.000000	Digital Wallet	Takeaway	03:36
4	TXN_3160411	Coffee	2.0	2.000000	4.000000	Digital Wallet	In-store	00:00
...	...	...	...	...	...	...	...	
9995	TXN_7672686	Coffee	2.0	2.000000	4.000000	Digital Wallet	Takeaway	03:36
9996	TXN_9659401	Juice	3.0	2.949984	3.000000	Digital Wallet	Takeaway	00:00
9997	TXN_5255387	Coffee	4.0	2.000000	8.000000	Digital Wallet	Takeaway	00:00
9998	TXN_7695629	Cookie	3.0	2.949984	3.000000	Digital Wallet	Takeaway	00:00
9999	TXN_6170729	Sandwich	3.0	4.000000	12.000000	Cash	In-store	00:00

10000 rows × 8 columns

In [12]:

```
# Quantity ko proper numeric banao
df["Quantity"] = pd.to_numeric(
    df["Quantity"],
    errors="coerce"
)

# Null hatao
quantity_data = df["Quantity"].dropna()

# Histogram
plt.figure(figsize=(8,5))

plt.hist(quantity_data)

plt.title("Quantity Distribution")
plt.xlabel("Quantity")
plt.ylabel("Frequency")

plt.show()
```



Quantity Distribution Analysis Graph Type

Histogram

Objective

The purpose of this histogram is to analyze the distribution of product quantities purchased by customers and identify purchasing patterns within the cafe sales dataset.

Observation

The histogram shows the frequency of transactions for different quantity values ranging from 1 to 5.

Quantity 3 has the highest frequency among all quantity levels. Quantities 2 and 5 are also frequently purchased by customers. Quantities 1 and 4 occur slightly less often compared to the other quantity values. The distribution appears relatively balanced across all quantity categories. Interpretation

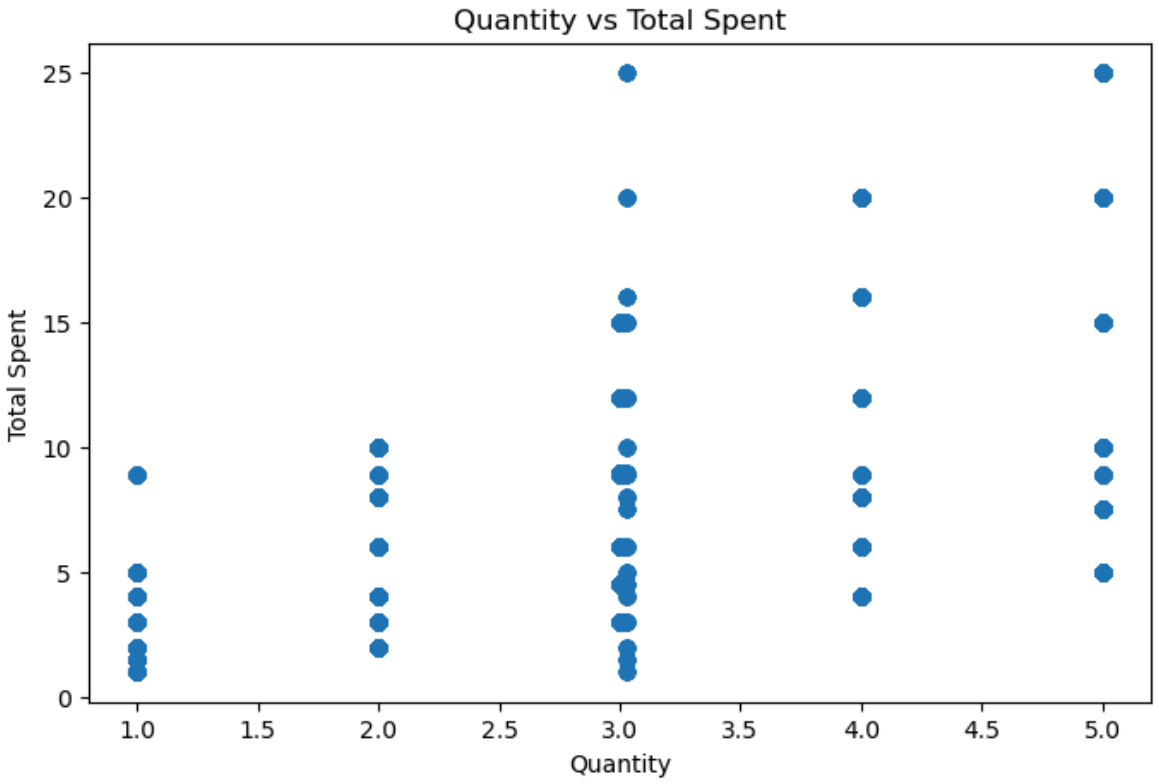
The results indicate that customers generally purchase between 1 and 5 units per transaction, with a slight preference for purchasing 3 units.

Since no quantity level dominates the dataset excessively, customer purchasing behavior appears consistent and evenly distributed.

Business Insight Quantity 3 represents the most common purchase volume and may indicate a typical customer order size. Inventory planning can be optimized by considering that customers frequently purchase multiple units rather than single-unit orders. The balanced distribution suggests stable demand patterns across different quantity levels. Conclusion

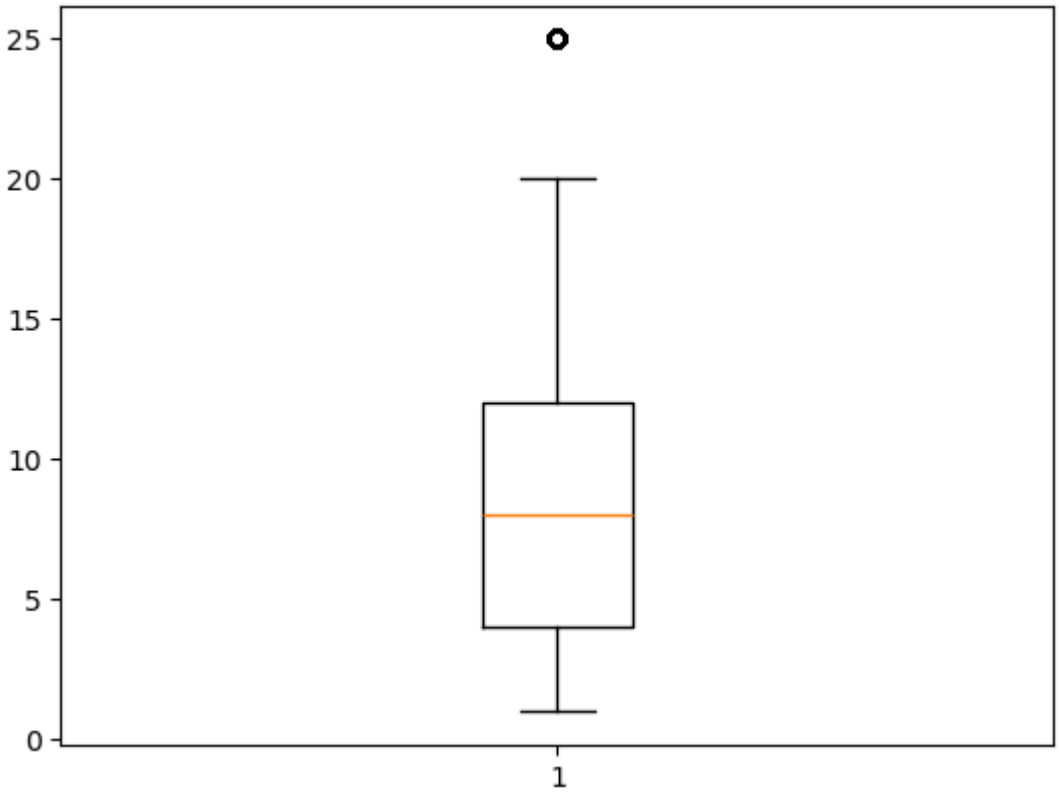
The Quantity Distribution histogram reveals that customer purchases are fairly evenly distributed across quantity values from 1 to 5, with Quantity 3 being the most frequently purchased amount. No significant outliers or unusual purchasing patterns were observed, indicating consistent customer buying behavior within the dataset.

In [13]:
```
scatter_df = df[
    ["Quantity", "Total Spent"]
].dropna()

plt.figure(figsize=(8,5))

plt.scatter(
    scatter_df["Quantity"],
    scatter_df["Total Spent"]
)

plt.title("Quantity vs Total Spent")
plt.xlabel("Quantity")
plt.ylabel("Total Spent")

plt.show()
```

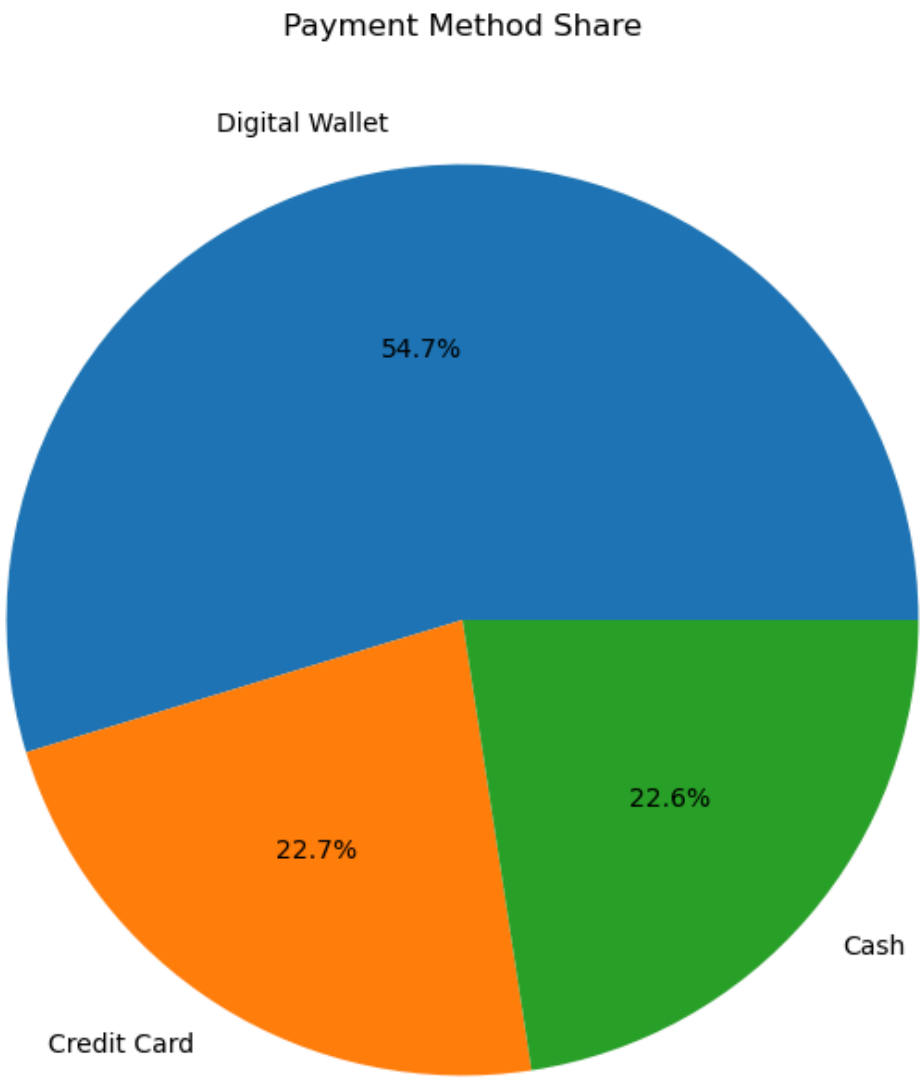


In [ ]:
```
```

In [14]:
```
plt.boxplot(
    pd.to_numeric(
        df["Total Spent"],
        errors="coerce"
    ).dropna()
)

plt.show()
```

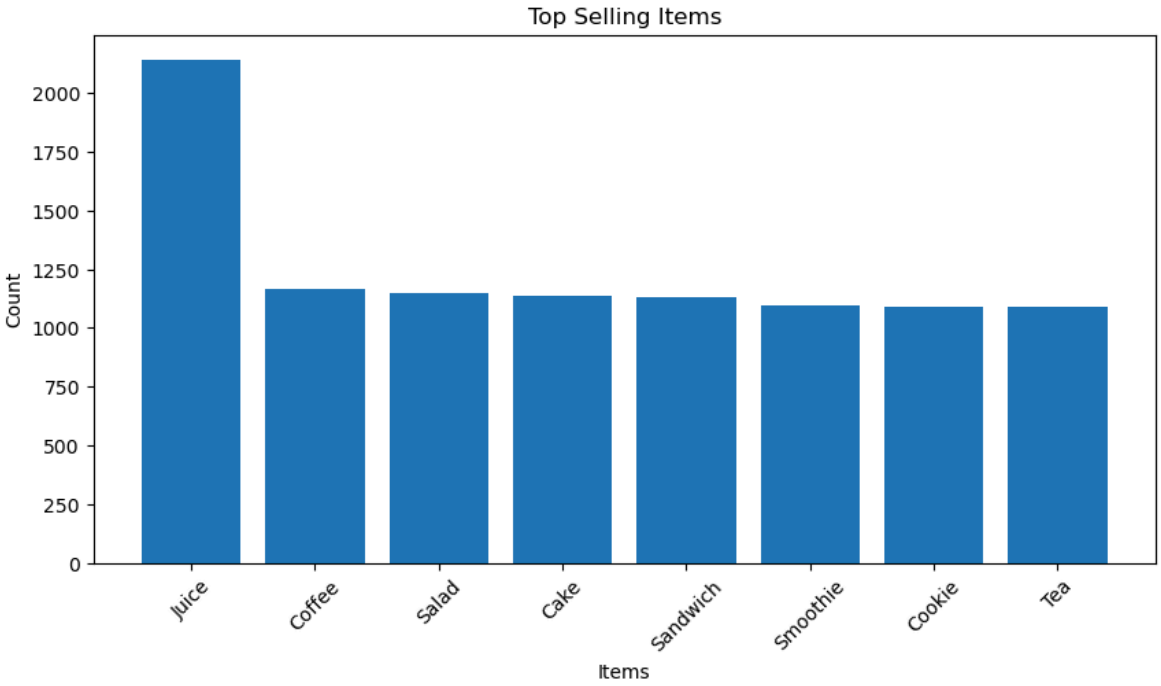


In [15]:
```
payment_counts = (
    df["Payment Method"]
    .astype(str)
    .replace("nan", pd.NA)
    .dropna()
    .value_counts()
)

plt.figure(figsize=(8,8))

plt.pie(
    payment_counts.values,
    labels=payment_counts.index,
    autopct="%1.1f%%"
)

plt.title(
    "Payment Method Share"
)

plt.show()
```



In [16]:
```
top_items = (
    df["Item"]
    .astype(str)
    .replace("nan", pd.NA)
    .dropna()
    .value_counts()
    .head(10)
)

plt.figure(figsize=(10,5))

plt.bar(
    top_items.index,
    top_items.values
)

plt.title(
    "Top Selling Items"
)

plt.xlabel("Items")
plt.ylabel("Count")
```

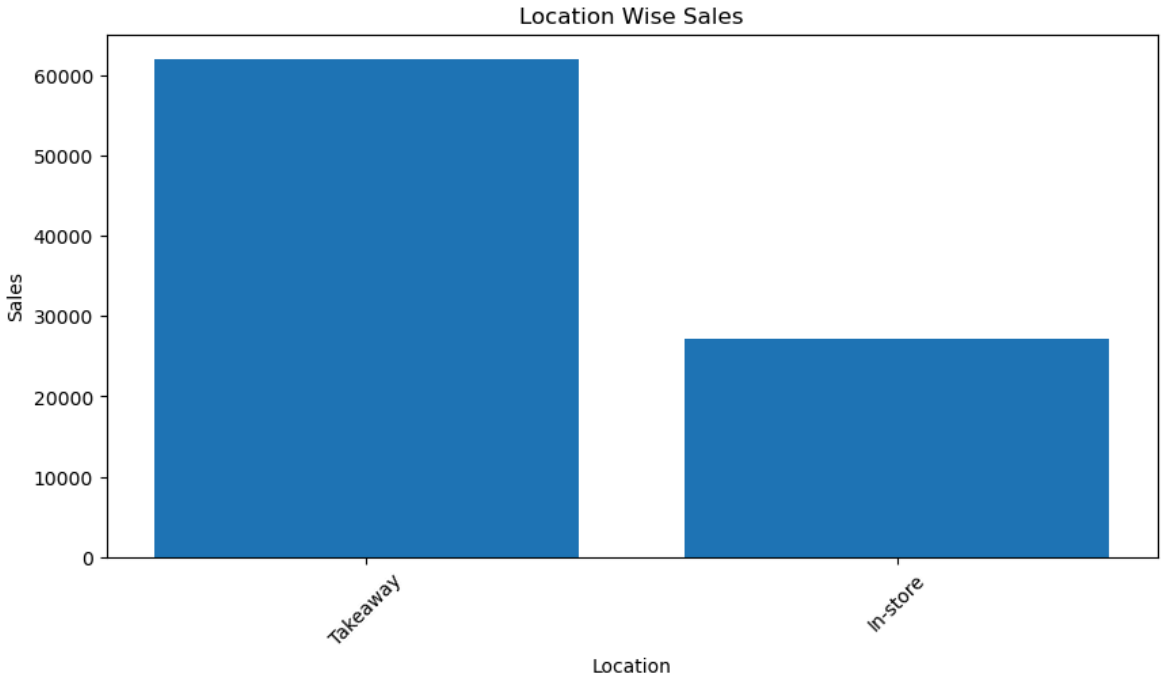
```
plt.xticks(rotation=45)

plt.show()
```



In [17]:
```
# Total Spent numeric
df["Total Spent"] = pd.to_numeric(
    df["Total Spent"],
    errors="coerce"
)

location_sales = (
    df.dropna(
        subset=[
            "Location",
            "Total Spent"
        ]
    )
    .groupby("Location")
    ["Total Spent"]
    .sum()
    .sort_values(
        ascending=False
    )
    .head(10)
)

plt.figure(figsize=(10,5))

plt.bar(
    location_sales.index.astype(str),
    location_sales.values
)

plt.title(
    "Location Wise Sales"
)

plt.xlabel("Location")
plt.ylabel("Sales")
```

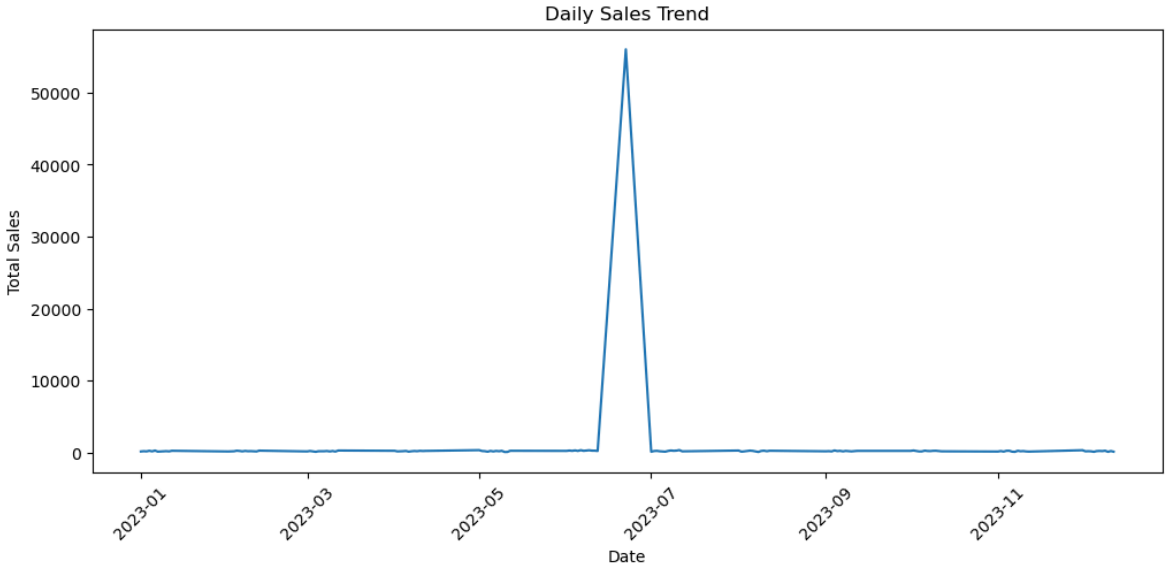
```
plt.xticks(rotation=45)

plt.show()
```

```
C:\Users\ASUS\AppData\Local\Temp\ipykernel_24372\222452848.py:14: FutureWarning:
The default of observed=False is deprecated and will be changed to True in a futu
re version of pandas. Pass observed=False to retain current behavior or observed=
True to adopt the future default and silence this warning.
  .groupby("Location")
```



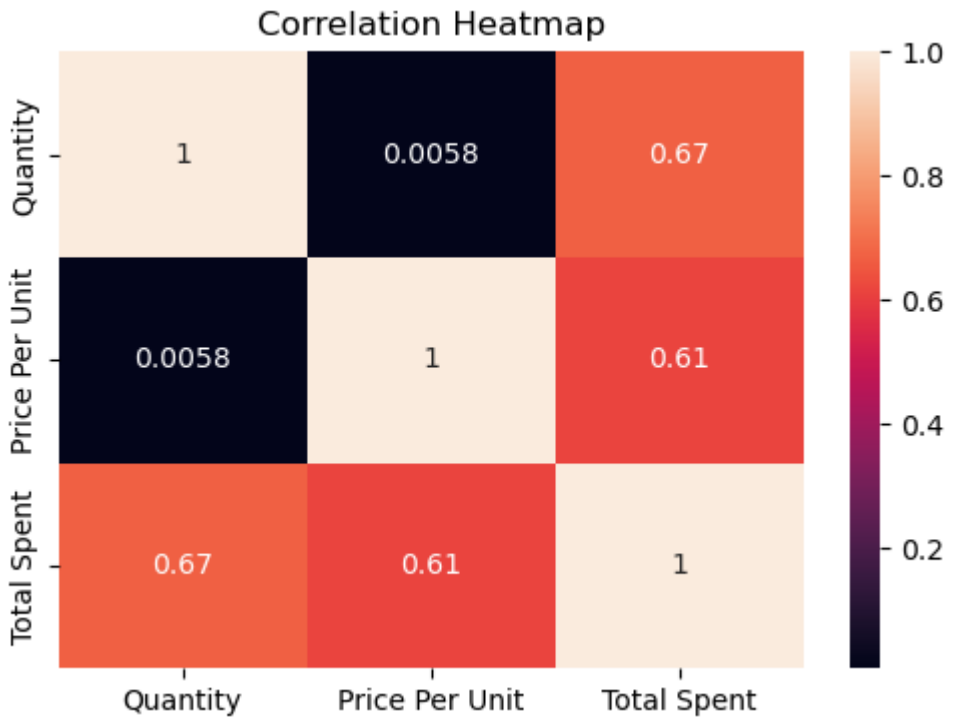
In [18]:
```
# Date fix
df["Transaction Date"] = pd.to_datetime(
    df["Transaction Date"],
    errors="coerce"
)

# Total Spent numeric
df["Total Spent"] = pd.to_numeric(
    df["Total Spent"],
    errors="coerce"
)

daily_sales = (
    df.dropna(
        subset=[
            "Transaction Date",
            "Total Spent"
        ]
    )
    .groupby(
        df["Transaction Date"]
        .dt.date
    )["Total Spent"]
    .sum()
)

plt.figure(figsize=(12,5))
```

```
plt.plot(
    daily_sales.index,
    daily_sales.values
)

plt.title(
    "Daily Sales Trend"
)

plt.xlabel("Date")
plt.ylabel("Total Sales")

plt.xticks(rotation=45)

plt.show()
```



In [19]:
```
# Numeric conversion
df["Quantity"] = pd.to_numeric(
    df["Quantity"],
    errors="coerce"
)

df["Price Per Unit"] = pd.to_numeric(
    df["Price Per Unit"],
    errors="coerce"
)

df["Total Spent"] = pd.to_numeric(
    df["Total Spent"],
    errors="coerce"
)

corr = df[
    [
        "Quantity",
        "Price Per Unit",
        "Total Spent"
    ]
].corr()

plt.figure(figsize=(6,4))
```

```
sns.heatmap(
    corr,
    annot=True
)

plt.title(
    "Correlation Heatmap"
)

plt.show()
```



In [ ]:

In [ ]:

In [ ]: